

Socket-Go Messaging Protocol

Design, Application-Layer Semantics, and Implementation

Suphachock Kamlek – 6710451321

Computer Communications Computer Science

August 30, 2026

Presentation roadmap

The problem: reliable messaging over a byte stream

What the application needs

- ▶ Route messages by topic.
- ▶ Support independent publishers and consumers.
- ▶ Deliver asynchronously over long-lived connections.
- ▶ Detect processing success with application-level ACK.
- ▶ Recover from slow consumers and lost sessions.

Why TCP alone is not enough

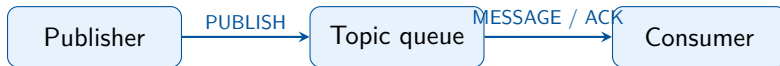
- ▶ TCP transports an ordered byte stream, not messages.
- ▶ A successful TCP write is not a successful business operation.
- ▶ The server still needs framing, routing, ACK, and retry rules.

Program objective

Application definition

Socket-Go is a topic-based messaging broker implemented as a custom application-layer protocol over TCP.

Publisher	Broker	Consumer
Sends a message to a topic.	Queues and routes messages.	Receives and acknowledges work.



Application characteristics

- ▶ **Asynchronous:** the publisher does not wait for consumer processing.
- ▶ **Topic-based:** routing is determined by a logical topic name.
- ▶ **Multiplexed:** several logical channels share one TCP connection.
- ▶ **At-least-once:** missing ACKs may cause redelivery.
- ▶ **Backpressure-aware:** pending queue and in-flight limits are bounded.
- ▶ **Concurrent:** sessions, Exchange, and Topics run independently.
- ▶ **Actor-owned state:** Exchange/Topic serialize state through commands.

Important consequence

Consumers must tolerate duplicate MessageID values and should perform application-level deduplication when the operation is not idempotent.

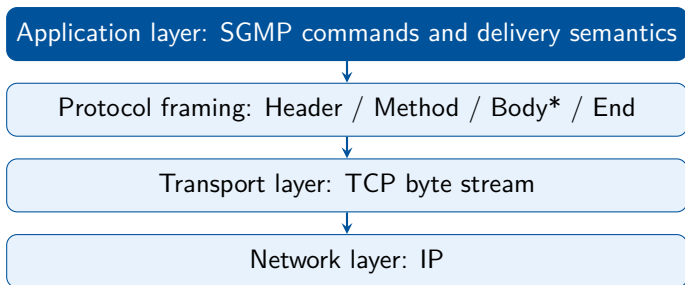
Why TCP instead of UDP?

Requirement	TCP	UDP
Ordered byte stream	Yes	No
Retransmission	Built in	Application must implement
Flow / congestion control	Built in	Not provided
Long-lived full-duplex session	Natural	Possible, but custom
Message-level ACK	Still needed	Still needed

Decision

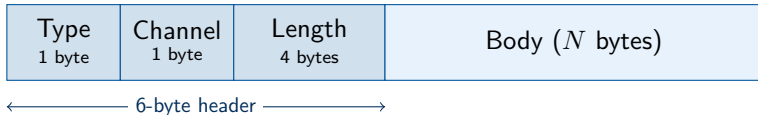
Use TCP for transport reliability, then add application semantics for framing, processing acknowledgement, retry, and routing.

Layered design



- ▶ TCP provides transport, not application messages.
- ▶ Framing makes message boundaries explicit.
- ▶ The application layer defines observable behavior and guarantees.

Wire format: one frame



- ▶ Integer fields use network byte order (big-endian).
- ▶ A frame body is limited to 1024 bytes.
- ▶ Larger logical payloads use multiple Body frames.

Wire format: frame types

0	Header	key=value metadata
1	Method	PUBLISH, ACK, PING, ...
2	Body	payload fragment
3	Heartbeat	standalone liveness frame
4	End	logical message terminator

Logical message grammar

Header Method Body* End

- ▶ Every frame in one logical message uses the same channel.
- ▶ The processor rejects invalid order or channel mismatch.
- ▶ Heartbeat is the only standalone frame sequence.

Logical channels are not Go channels

Protocol meaning

A channel is a logical stream multiplexed inside one TCP connection.

- ▶ Channel ID is carried in every frame.
- ▶ The server associates a channel with one session.
- ▶ A channel has a role: publisher or consumer.

TCP connection

channel 1: consumer / control
channel 2: publisher

SUBSCRIBE -> ConsumerChannel
PUBLISH -> PublisherChannel
MESSAGE -> ConsumerChannel
ACK -> ConsumerChannel

Protocol rule

A channel **MUST NOT** change role after it has been established.

Command model

Command	Direction	Meaning
PUBLISH	Client → Server	Append a message to a topic queue
SUBSCRIBE	Client → Server	Register a consumer channel
UNSUBSCRIBE	Client → Server	Remove a topic subscription
MESSAGE	Server → Client	Deliver one message
ACK	Client → Server	Confirm successful processing
NACK	Client → Server	Reject and requeue a delivery
TOPICS	Client → Server	Query topic names
PING / PONG	Both directions	Liveness check
EXIT / BYE	Both directions	Graceful connection close

Message identity and delivery identity

Message identity

- ▶ MessageID identifies the original message.
- ▶ It remains unchanged across retries.
- ▶ It is the key for consumer deduplication.

Delivery identity

- ▶ DeliveryID identifies one delivery attempt.
- ▶ It is the identifier carried by ACK/NACK.
- ▶ A retry creates a new DeliveryID.

MessageID = M1 $\Rightarrow$ DeliveryID = D1, attempt=1 $\Rightarrow$ DeliveryID = D2, attempt=2

Publish path

- ❶ The client sends `PUBLISH` on a `PublisherChannel`.
- ❷ Session and `FrameProcessor` assemble one `FullFrame`.
- ❸ Process validates channel ownership and message size.
- ❹ Exchange resolves the topic.
- ❺ Topic appends the message to its bounded pending queue.
- ❻ The server returns `PUBLISH_OK` after queue acceptance.

Meaning of `PUBLISH_OK`

The broker accepted the message; it does not mean that a consumer has finished processing it.

Delivery path



- ▶ Topic chooses an eligible consumer using round-robin order.
- ▶ The delivery is inserted into `unacked` before further progress.
- ▶ A consumer receives `MESSAGE` and ACKs after processing.

ACK, NACK, timeout, and retry

ACK

- ▶ Validate channel ownership.
- ▶ Remove DeliveryID from `unacked`.
- ▶ Decrease in-flight count.
- ▶ Release capacity for the next message.

`pending` → `unacked` → ACK

NACK / timeout

- ▶ Remove the old delivery.
- ▶ Requeue the same MessageID.
- ▶ Increment attempt and create a new DeliveryID.
- ▶ Drop after the retry limit is reached.

`pending` ← timeout / NACK

Backpressure and max in-flight

Definition

`maxInFlight` is the maximum number of deliveries sent to one consumer that have not received ACK yet.

```
maxInFlight = 1
```

```
send A  
wait ACK(A)  
send B
```

```
maxInFlight = 32
```

```
send A ... send Z  
pause when window is full  
continue after ACK arrives
```

- ▶ The pending queue is bounded; a full queue returns `QUEUE_FULL`.
- ▶ A slow consumer does not receive unlimited work.

Connection close and failure behavior

- ① A session detects EOF, write failure, or explicit `EXIT`.
- ② The server removes all subscriptions owned by that session.
- ③ Unacknowledged deliveries are requeued.
- ④ Consumer outbound buffers are closed by their owner.
- ⑤ Other consumers may receive the requeued messages.

Reliability boundary

TCP protects the byte stream. The application protocol protects message processing semantics through ACK, retry, and requeue.

Implementation map

Component	Responsibility
<code>internal/protocol/frame.go</code>	Encode and decode one wire frame
<code>frame_processor.go</code>	Validate and assemble logical messages
<code>internal/server/tcp.go</code>	Accept sessions and serialize writes
<code>internal/core/process.go</code>	Own a session's channel registry
<code>internal/core/exchange.go</code>	Own topic registry and route commands
<code>internal/core/topic.go</code>	Own queue, subscriptions, ACK, retry, and delivery
<code>internal/core/delivery.go</code>	Define message and delivery metadata
<code>cmd/server/main.go</code>	Bind protocol requests to core services
<code>cmd/client/main.go</code>	CLI and automatic ACK for MESSAGE

Frame processor state machine

```
switch {  
case fp.stage == stageStart && f.Type == HeaderType:  
    fp.stage = stageMethod  
case fp.stage == stageMethod && f.Type == MethodType:  
    fp.stage = stageBody  
case fp.stage == stageBody && f.Type == BodyType:  
    fp.bodyLen += f.Length  
case fp.stage == stageBody && f.Type == EndType:  
    fp.stage = stageComplete  
default:  
    return ErrInvalidFrameSequence  
}
```

- ▶ A single processor instance assembles one logical message at a time.
- ▶ Channel mismatch is rejected before the message reaches routing.
- ▶ Body length is checked against the logical message limit.

Exchange as a command actor

```
type Exchange struct {
    ID      uuid.UUID
    commands chan ExchangeCommand
    topics   map[string]topicHandle
}

func (e *Exchange) Run(ctx context.Context) error {
    for {
        select {
        case <-ctx.Done():
            return nil
        case command := <-e.commands:
            result := e.handle(ctx, e.topics, command)
            if command.Reply != nil {
                command.Reply <- result
            }
        }
    }
}
```

- ▶ Topic registry is private to the Exchange actor.
- ▶ Callers submit commands and wait on a reply channel.
- ▶ Routing does not mutate Topic state directly.

Topic owns the message lifecycle

```
type Topic struct {
    Name      string
    channels  map[uuid.UUID]*Channel
    order     []uuid.UUID
    inFlight  map[uuid.UUID]int
    unacked   map[uuid.UUID]*DeliveryMessage
    pending   []*RawMessage
    rrIdx     int
}

func (t *Topic) drain() {
    for len(t.pending) > 0 {
        if !t.tryDeliver(t.pending[0]) {
            return // no eligible consumer: apply backpressure
        }
        t.pending = t.pending[1:]
    }
}
```

- ▶ The Topic goroutine is the owner of queue and delivery state.
- ▶ Delivery is attempted only when a consumer has available credit.

Delivery metadata and automatic ACK

```
type DeliveryMeta struct {  
    Topic      string  
    MessageID  uuid.UUID  
    DeliveryID uuid.UUID  
    SendChannelID uuid.UUID  
    RecvChannelID uuid.UUID  
    Attempt    int64  
    Deadline   time.Time  
}
```

```
if message.Method == "MESSAGE" {  
    frames, err := buildAckFrames(message)  
    if err == nil {  
        _ = conn.SendMessage(frames...)  
    }  
}
```

- ▶ The CLI acknowledges after it has received the MESSAGE.
- ▶ A production consumer should ACK after its business operation succeeds.

Server request routing

```
switch message.Method {
case "SUBSCRIBE":
    channel, _ := state.channel(session, message.Channel,
        core.ConsumerChannel)
    err = exchange.Subscribe(ctx, topic, channel)
case "PUBLISH":
    channel, _ := state.channel(session, message.Channel,
        core.PublisherChannel)
    err = exchange.PublishFrom(ctx, topic, channel.ID, payload)
case "ACK":
    err = exchange.Ack(ctx, topic, channel.ID, deliveryID)
}
```

- ▶ The handler validates the protocol boundary.
- ▶ Exchange remains the only routing entry point.
- ▶ Topic remains the only message lifecycle owner.

Why the design avoids core data races

- ▶ Exchange registry is read and modified by the Exchange goroutine.
- ▶ Each Topic owns its pending queue, subscribers, and unacked map.
- ▶ Commands cross boundaries through channels with reply channels.
- ▶ A Session serializes writes so multiple delivery sources cannot interleave frames.
- ▶ The Process registry uses a small lock because session lifecycle and lookup may overlap.

Design boundary

The no-mutex goal applies to core message state; network output still needs one serialization point.

Testing strategy

Unit tests

Frame validation, command parsing, queue state, ACK, retry, and role validation.

Integration tests

Real TCP connection with subscribe, publish, MESSAGE, and ACK.

Concurrency checks

Race detector and static analysis validate goroutine interactions.

```
go test ./...  
go test -race ./...  
go vet ./...
```

Scenarios covered by the test suite

- ▶ Message is delivered and removed from `unacked` after ACK.
- ▶ Messages wait in the queue until a consumer subscribes.
- ▶ Round-robin distributes messages across consumers.
- ▶ `maxInFlight` pauses delivery until ACK releases capacity.
- ▶ Timeout retries preserve `MessageID` and increment attempt.
- ▶ NACK requeues immediately.
- ▶ ACK from the wrong channel is rejected.
- ▶ Queue capacity returns `ErrQueueFull`.
- ▶ Unsubscribe requeues outstanding deliveries.
- ▶ TCP integration verifies the complete publish/subscribe/ACK path.
- ▶ CLI reader automatically sends ACK for MESSAGE.

What the measurements should answer

- ① How many requests per second can the broker accept?
- ② How does throughput change with multiple publishers?
- ③ How does `maxInFlight` affect latency and memory?
- ④ How much overhead is added by framing and ACK?
- ⑤ What happens when consumers are slower than publishers?

Interpretation

Performance results should be reported together with payload size, concurrency, ACK behavior, queue limits, and whether the test is end-to-end.

Current scope and future extensions

Current implementation

- ▶ In-memory topics and queues.
- ▶ At-least-once delivery.
- ▶ Single broker process.
- ▶ Bounded queue and retry limit.
- ▶ TCP transport with custom framing.

Future work

- ▶ Durable / persistent queues.
- ▶ Dead-letter queue.
- ▶ Authentication and TLS policy.
- ▶ Explicit channel-open handshake.
- ▶ Consumer groups and distributed routing.
- ▶ Metrics and load-shedding policy.

Final architecture



Central design idea

Keep routing in Exchange, keep message lifecycle in Topic, and keep TCP ownership in Session. Each boundary communicates through explicit commands.

Conclusion

- ▶ Socket-Go is an application-layer messaging protocol over TCP.
- ▶ Custom framing converts a byte stream into logical messages.
- ▶ Topic actors provide queueing, delivery, ACK, retry, and backpressure.
- ▶ At-least-once semantics improve recovery but require duplicate tolerance.
- ▶ The design is ready for separate live demonstration and benchmark discussion.

Thank you

Questions?